

## **Artie Shevchenko**

Ex ingeniero de software de Google, ex CEO y fundador de un startup. Actualmente, es ingeniero de software en Canva y profesor en la Universidad ITMO, que es famosa por ganar más copas mundiales de programación que cualquier otra universidad en el mundo.

También es el autor del libro recientemente publicado *Code Health Guardian, the Old New Role of Human Programmers in the AI Era*.

### **¿Qué se necesita para que los programadores tengan éxito en la era de la IA?**

Según él, hay tres habilidades clave:

- Una es colaborar con la IA.
- Otra es delegar en la IA.

Opina que lo mejor es convertirse en un experto en la gestión de la complejidad del código, simplemente porque el razonamiento complejo es difícil para la IA y, en ingeniería de software, ese es probablemente el problema más difícil que tenemos.

Por eso propone un nuevo rol: el **Code Health Guardian**, el guardián de la salud del código.

### **¿Por qué le cuesta a la IA mantener las cosas simples?**

La IA tiene dificultades con el razonamiento verdaderamente complejo. Aunque muchas veces ofrece respuestas que parecen claras y simples, esa simplicidad suele ser superficial.

En sistemas grandes:

- No entiende completamente el contexto histórico.
- No anticipa efectos secundarios profundos.
- No gestiona bien la evolución del sistema en el tiempo.

La gestión real de la complejidad sigue siendo una ventaja humana.

### **La salud del código**

No es solo simplicidad o mantenibilidad, sino, en el fondo, una cuestión de productividad.

El verdadero reto está en cada individuo: encontrar el equilibrio entre productividad a corto plazo y productividad a largo plazo.

Si uno se enfoca demasiado en el largo plazo, pasa el tiempo planificando sin entregar. Si se enfoca demasiado en el corto plazo, genera complejidad que dificulta el trabajo futuro.

Por eso, la clave está en mantener un equilibrio saludable: entregar valor ahora sin comprometer la capacidad de evolucionar el sistema en el futuro.

El verdadero desafío está en encontrar el equilibrio adecuado según el contexto.

### **Los 3 Problemas de la Complejidad**

#### **1. Change Amplification (Amplificación de cambios)**

Ocurre cuando un cambio que debería ser pequeño termina afectando cientos de líneas de código y múltiples archivos.

Esto sucede cuando el sistema está altamente acoplado o mal estructurado. Aunque es considerado el menos grave de los tres problemas, impacta directamente en la velocidad de desarrollo y puede ralentizar significativamente la evolución del sistema.

Este problema se vuelve más crítico a medida que el sistema crece, ya que cada modificación requiere revisar múltiples partes del código, aumentando el riesgo de introducir errores.

#### **2. Cognitive Load (Carga cognitiva)**

Se refiere al esfuerzo mental necesario para comprender el código.

Una alta carga cognitiva implica que los desarrolladores necesitan invertir muchas horas simplemente para entender qué está ocurriendo antes de poder realizar cambios. Esto no solo reduce la productividad, sino que también incrementa la probabilidad de errores y genera agotamiento.

#### **3. Unknown Unknowns (Lo que no sabes que no sabes)**

Es el problema más grave. Ocurre cuando falta información esencial y el desarrollador ni siquiera es consciente de que esa información es necesaria. Esto suele deberse a ausencia de documentación, decisiones antiguas sin explicación o configuraciones complejas cuyo propósito no está claro.

En el ejemplo mencionado en la conversación, la eliminación de una configuración antigua provocó el fallo masivo de procesos automáticos porque no estaba documentado por qué esa configuración era necesaria. Este tipo de situaciones genera errores graves y difíciles de anticipar.

### **Relación con la Productividad**

Los tres problemas están interconectados y pueden generar ciclos negativos: más complejidad produce más errores, los errores generan más parches, y los parches

aumentan aún más la complejidad. Además, la complejidad contribuye al agotamiento de los desarrolladores, lo que impacta directamente en la calidad del producto.

## **Causas Principales de la Complejidad**

Se habla de que hay 7 pero destacan 3 en el podcast.

**1. Obscurity (Oscuridad)** Es la causa más importante. Se manifiesta cuando el código es difícil de entender debido a nombres poco claros, falta de comentarios o estructuras difíciles de rastrear. La oscuridad incrementa los unknown unknowns y complica el mantenimiento.

## **2. Interfaces Complejas**

Las interfaces con demasiados parámetros, responsabilidades poco cohesionadas o excesivamente genéricas generan confusión. Una interfaz puede parecer simple al leerla, pero ser difícil de usar correctamente. Diseñar APIs demasiado flexibles sin considerar los casos de uso reales puede añadir complejidad innecesaria y aumentar tanto la carga cognitiva como la amplificación de cambios.

## **3. Unfamiliarity (Falta de familiaridad)**

La falta de consistencia en la forma de estructurar el código dentro de un equipo incrementa la dificultad de mantenimiento. Cuando cada desarrollador sigue estilos o patrones distintos, comprender el sistema requiere un esfuerzo adicional.

Mantener consistencia dentro de un equipo facilita el trabajo conjunto, permite mejorar sistemas de forma global y reduce errores derivados de malentendidos.

## **Conclusión**

La complejidad en el software no es un único problema, sino un conjunto de factores interrelacionados que afectan tanto al producto como al equipo. Comprender estos problemas y sus causas es fundamental para mantener un sistema sostenible a largo plazo y evitar ciclos negativos de degradación del código.

## **Design by contract.**

Básicamente se dice que un módulo, entendiéndose este como la interfaz más su implementación ha de tener contratos claros. Así, se pueden diseñar los componentes dando por supuesto que se cumplirán ciertas condiciones de entrada (precondiciones), mientras que se deberán garantizar ciertas condiciones de salida (postcondiciones), así como la invariante de clase (propiedades que se mantienen invariantes a pesar del procesamiento realizado por el componente). Se destaca que no basta con realizar test ya que también son recomendables el uso de assertions en runtime para validar precondiciones y fallar rápido. Esto es debido a que una vez llegado a estado de producción hay millones de estados posibles y los test no lo

cubren todo y con los assertions, evitas meter lógica extra para inputs que nunca debería de ser válidos.

Cierra este principio con una recomendación que se está convirtiendo en aprendizaje cultural, que es el hecho de no tragarse excepciones por defecto, ya que en muchos/mayorías de casos es mejor fallar explícitamente que esconder errores.

### **DRY (Don't Repeat Yourself) pero desde la perspectiva de la complejidad.**

Cuando se habla de esta parte en el podcast, Artie tiene un matiz importante que menciona. Y es el hecho de que la duplicación de código abarca 3 tipos distintos de complejidades: change amplification, cognitive load y unknown unknown. Dice que la gente se obsesiona con eliminar cada duplicado cuando es mejor ver el DRY como algo pragmático.

Se recomienda siempre eliminar los unknown unknown primero, luego evaluar el coste-beneficio de abstraer el código duplicado y por último eliminar duplicación si realmente aporta simplicidad.

Nombra algunos ejemplos usados en Google para evitar encontrarse con unknown unknown cuando apliques DRY como es el simple hecho de hacer comentarios del tipo: “// If you change this logic, update XYZ in frontend” o “// Duplicate of validation in ServiceA”

### **Procesos para mantener código sano.**

#### **Design Docs.**

Dice que no hace falta que sea un “big upfront design”(gran diseño inicial) ya que con un diseño mínimo pero consciente antes de implementar es suficiente y que no tiene por qué ser anti-agile .

Son documentos de planificación livianos escritos antes de implementar el código para solicitar feedback y así reducir el retrabajo o evitar borrar código ya implementado. Suelen contener contexto, objetivos, vista general de la solución y sobretodo y más importante, trade-off analysis que es básicamente el proceso de discutir los pros y contras de diferentes opciones para solucionar requisitos.

#### **Revisión de Código.**

Habla de que en las revisiones de código no hay que centrarse solo en la calidad del código, si no que realmente propone 4 objetivos:

- Calidad de código: detección de fallos, mejora de diseño etc.
- Compartir conocimiento: el revisor ve el contexto real(código) y se comparte aprendizaje con base concreta.

- Velocidad de entrega: optimizar revisiones para no caer en perfeccionismo.
- Seguridad psicológica: que haya seguridad psicológica fomenta el clima seguro lo que permite reconocer errores facilitar el aprendizaje y aumentar productividad del equipo.